```
  </li>
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="Identity"
      asp-page="/Account/Login">Login</a>
  </li>
}
</ul>
```

❶ Updates to your project's namespace that contains ApplicationUser

❷ The default template uses IdentityUser. Update to use ApplicationUser instead.

This partial shows the current login status of the user and provides links to register or sign in. All that remains is to render the partial by calling

```
<partial name="_LoginPartial" />
```

in the main layout file of your app, _Layout.cshtml.

And there you have it: you've added Identity to an existing application. The default UI makes doing this relatively simple, and you can be sure you haven't introduced any security holes by building your own UI!

As I described in section 23.3.4, there are some features that the default UI doesn't provide and you need to implement yourself, such as email confirmation and MFA QR code generation. It's also common to find that you want to update a single page here and there. In the next section I'll show how you can replace a page in the default UI, without having to rebuild the entire UI yourself.

23.5 Customizing a page in ASP.NET Core Identity's default UI

In this section you'll learn how to use scaffolding to replace individual pages in the default Identity UI. You'll learn to scaffold a page so that it overrides the default UI, allowing you to customize both the Razor template and the `PageModel` page handlers.